FAQ-19: How Do I Bring a Third-Party SaaS Platform's Telemetry Into Dynatrace?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 19 — Integrating Third-Party SaaS Telemetry | **Created:** July 2026 | **Last Updated:** 11/06/2026

Overview

Sooner or later a platform you do not run becomes something you have to answer for. A secure-access service sits between every user and every application. A CDN fronts your customer traffic. An identity provider gates every login. A payment gateway carries revenue. None of them will ever host a OneAgent, and none of them will appear in Smartscape on their own — yet when they degrade, the tickets land on your team.

These platforms all present the same shape of problem, and it is not the shape most integration guides assume. The vendor does not publish one feed; it publishes **several, in different formats, over different transports, with different retention implications** — and the most operationally valuable signal is usually the one that is not a log at all. The temptation is to point all of it at Grail and sort it out later. That is the expensive answer, and it is expensive in two directions at once: storage cost for records nobody queries, and a compliance surface for user identities and destinations you did not intend to retain.

This entry covers **the reusable pattern**: how to choose an ingestion route from the vendor's transport rather than your preference, how to run signal-first so raw records are processed and then discarded, how to model the vendor's objects as real Smartscape topology, and how to isolate what you do keep. It is deliberately vendor-neutral — **FAQ-20** applies every section of it to Zscaler as a worked example; **FAQ-23** (Box) and **FAQ-27** (Microsoft 365) do the same for audit feeds.

A note on scope. This is an ingestion-and-modeling entry. Once the data is in Grail with topology attached, the downstream work is ordinary and already documented elsewhere in this corpus — § 8 routes you there rather than restating it.

Table of Contents

1. Short Answer
2. Three Signal Classes, Three Different Answers
3. Choosing the Ingestion Route
4. Signal-First Ingestion — Extract, Then Discard
5. Modeling the Vendor's Objects as Topology
6. Isolating What You Do Retain
7. Normalizing Dimensions Across Feeds
8. Where This Plugs Into the Rest of the Corpus
9. Recommended Approach
10. Common Gotchas

Prerequisites

Requirement	Details
Dynatrace environment	SaaS with **Grail** and **OpenPipeline**. The topology work in § 5 depends on Smartscape on Grail; the extract-then-discard pattern in § 4 depends on OpenPipeline stages
Vendor-side access	Admin rights on the source platform to configure log streaming and to mint API credentials. In most organizations this is a different team — budget for the hand-off
Permissions	Enough access to create OpenPipeline pipelines and routes, create Grail buckets, and assign bucket permissions
A security decision-maker	§ 4 and § 6 are retention and exposure decisions, not engineering ones. Identify who signs off before you build the pipeline, not after
Related series	**OPIPE** and **OPLOGS** (OpenPipeline depth), **ORGNZ** (buckets, segments, retention), **FAQ-09** (metric-vs-log query economics), **FAQ-15** (DPL, for parsing non-JSON feeds), **FAQ-16** (Smartscape query patterns), **DASH** / **ALERT** / **SLO** (downstream consumption)

1. Short Answer

Let the vendor's transport pick your ingestion route, extract the signal at ingest and discard the raw record by default, and model the vendor's operational objects as `CUSTOM_*` Smartscape nodes so everything downstream inherits topology.

Four decisions carry almost all the weight, and they are best made in this order:

Decision	The rule
1. Route	Determined by what the vendor can *send*, not by what you would prefer to receive. An HTTPS-push feed can reach Dynatrace directly; a raw-TCP or syslog feed cannot, and needs a collector hop. See § 3
2. Retention	Default to *discard*. Extract the metric, event, or alert you actually query, then drop the record. Keep raw only where security or compliance explicitly asks for it. See § 4
3. Topology	Model the vendor's objects — the applications, the connectors, the edges, the regions — as `CUSTOM_*` Smartscape nodes. This is the decision that makes every later dashboard and alert better. See § 5
4. Isolation	Whatever you do retain goes in a **dedicated bucket** with its own permissions, not in `default_logs`. See § 6

> **Sources:** Processing in OpenPipeline (DT docs), Smartscape node and edge extraction in OpenPipeline (DT docs). **Derived:** the four-decision ordering is this entry's framing — the platform imposes no order, but route constrains retention options and topology depends on fields that survive processing.

2. Three Signal Classes, Three Different Answers

The single most common mistake is treating a vendor integration as a *log* integration. In community practice, most SaaS platforms of this kind emit three distinct classes of signal, and they answer genuinely different questions — the split below is a framing for deciding

what to ingest, not a vendor-defined taxonomy, so check it against your own platform's catalogue. Deciding which questions you need answered — before you configure anything — is what keeps the project from becoming a log-volume problem.

Signal class	What it is	What it answers well	What it cannot answer
Transaction / access logs	Per-request or per-session records the platform writes as it handles traffic	Who did what, when, to which destination, with what outcome, how many bytes, allowed or denied	How it *felt* to the user. These are the platform's own view, recorded at its own vantage point
Infrastructure / component metrics	Health and capacity of the vendor's own components — gateways, connectors, collectors	Is the vendor-side infrastructure saturated, disconnected, or unbalanced	Anything about a specific user's session
Experience / probe telemetry	Active measurement from the endpoint or from a probe — device health, path quality, synthetic fetches	Was it slow, where in the path, and was the endpoint itself the problem	Full transaction detail; probes sample, they do not record everything

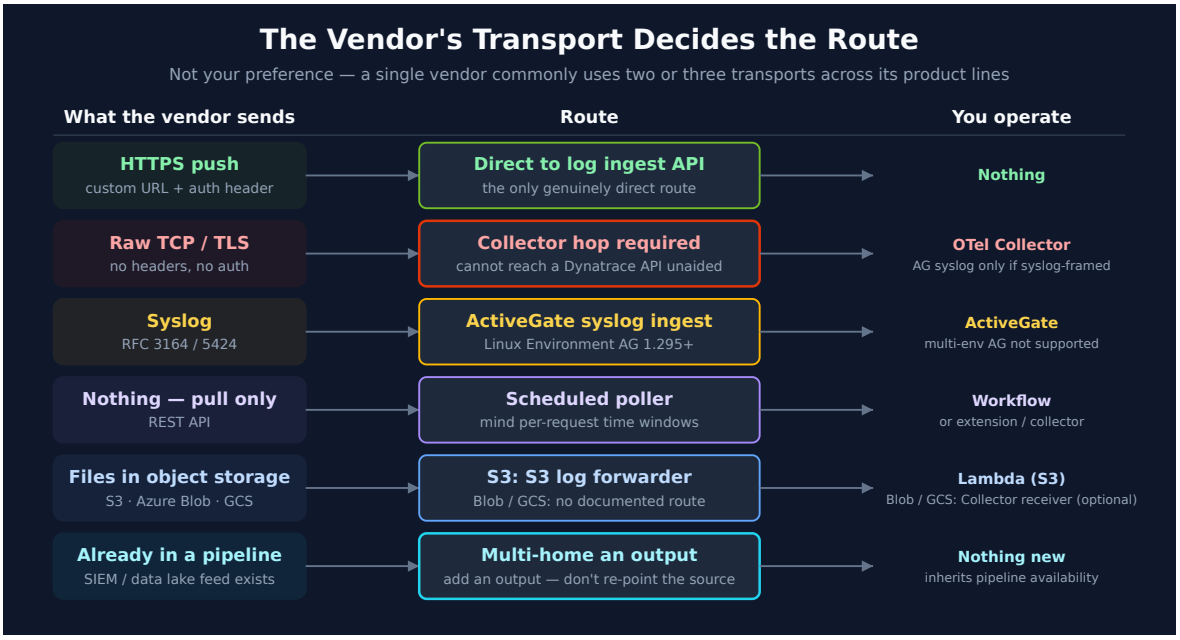
The practical consequence. Access logs are usually the largest, cheapest-to-obtain, and *least* directly useful for experience questions. Experience telemetry is usually the smallest, hardest to obtain (it is often a separate SKU and a separate API), and the one that actually answers "why is the user complaining." If your integration ships only the logs because the logs were the easy feed, you will have spent the storage budget on the signal class that answers the fewest of your questions.

Ask the question first. Write down the five questions the integration must answer, map each to a signal class using the table above, and configure only the feeds those classes require. Feeds are easy to add later; retention decisions made by accident are hard to unwind.

---

3. Choosing the Ingestion Route

This is where most integration plans go wrong, and the failure is nearly always the same one: assuming every feed from a vendor can be pointed at the Dynatrace log ingest endpoint. **It is the vendor's transport that decides the route, and a single vendor commonly uses two or three different transports across its own product lines.**



What the vendor can send	Route into Dynatrace	Notes
HTTPS POST to an arbitrary URL with custom headers	**Direct** to the Dynatrace log ingest API	The only genuinely direct route. Requires the vendor to support a custom endpoint *and* a custom `Authorization` header. Where it cannot set the header, a collector must terminate the push — § 3.3
Raw TCP / TLS to a host and port	**Collector hop required** — an OpenTelemetry Collector with a TCP or log receiver, or an existing pipeline product. The ActiveGate syslog receiver applies only if the stream is RFC 3164 / 5424 syslog	The vendor cannot authenticate to a Dynatrace API over raw TCP. Something must terminate the stream and forward it

What the vendor can send	Route into Dynatrace	Notes
Syslog (RFC 3164 / 5424)	**ActiveGate syslog ingestion** or an OTel Collector `syslog` receiver	Environment ActiveGate on Linux, 1.295+, using its embedded OTel Collector. Multi-environment ActiveGates do not support syslog ingestion
Nothing — pull only (REST API)	**Scheduled poller** — a Dynatrace workflow, an extension, or a collector	Common for experience/score APIs. Watch for per-request time-window limits, which set your minimum poll frequency. A generic Collector poller exists — check it against § 3.3 first
Files written to object storage — S3, Azure Blob Storage, Google Cloud Storage	**S3:** the Dynatrace **S3 log forwarder**, a Lambda function you deploy in your AWS account. **Azure Blob / GCS:** no documented Dynatrace route for files at rest	The S3 forwarder takes generic text logs and JSON streams, not only AWS service logs, and drops records older than 24 hours. Dynatrace's Azure and Google Cloud log forwarding read Event Hubs and Pub/Sub, not stored objects. For Blob and GCS, the Collector receivers in § 3.3 are an optional complement
Already flowing to a pipeline product you run	**Multi-home** an additional output to Dynatrace	Usually the cleanest enterprise answer — see below

> **Sources:**
>
> - Syslog ingestion with ActiveGate (DT docs) — *"Environment ActiveGate version 1.295+ on Linux installed to monitor remote technologies"*, which *"uses an embedded Dynatrace OpenTelemetry Collector instance"*; note also *"Multi-environment ActiveGates do not support syslog ingestion."* The receiver is scoped to syslog: *"Use this configuration only for syslog ingestion."*
> - Ingest syslog data with the OTel Collector (DT docs)
> - Cribl via HTTP (Dynatrace Hub)
> - Cribl via OpenTelemetry (Dynatrace Hub)
> - Syslog via OpenTelemetry Collector (Dynatrace Hub) — the listed collector-hop routes for feeds that cannot post directly. The two Cribl listings read *"Provided by Cribl"* and *"Support By the partner"*; the syslog collector listing reads *"Provided by Dynatrace"*

> ˜ Ingest AWS logs from Amazon S3 (DT docs) — the forwarder runs as *"an event-driven AWS Lambda function that you deploy in your own AWS account"*; *"Additionally, you can forward and ingest generic text logs and JSON streams"*; *"Log records with timestamps older than 24 hours are dropped at ingest."*
>
> ˜ Cloud provider log forwarding (DT docs) — *"Azure log forwarding allows you to stream Azure logs from Azure Event Hubs into Dynatrace logs via an Azure Function App instance."*; for Google Cloud, *"During setup, a new Pub/Sub subscription will be created."*
>
> ˜ **Docs check (10/29/2026):** the cloud provider log forwarding page, the Azure log forwarder setup page and the S3 forwarder page were read for a route that ingests objects already stored in Azure Blob Storage or Google Cloud Storage. None was found (control: the same pages document the S3 route). The Azure forwarder deploys a storage account as part of its own resources, not as a log source

3.1 The multi-home pattern — usually the right enterprise answer

If the vendor's logs already flow somewhere — a SIEM, a security data lake, an observability pipeline — you are rarely the first consumer, and you should not try to become the only one. Adding a **second output** from the existing pipeline is almost always better than re-pointing the source:

- **It does not disturb the security pipeline.** Re-pointing a source feed that a SOC depends on is a change with a blast radius well outside observability. Adding an output is not.
- **It gives you a transformation point you control.** Masking, field reduction, and sampling can be applied to the observability copy without touching the security copy, which usually must stay complete.
- **It solves the transport problem for free.** The pipeline product already terminates whatever exotic transport the vendor uses; its outputs are ordinary HTTP.

The Hub lists partner-provided destinations for at least one common pipeline product — Cribl, over both HTTP and OpenTelemetry. Both listings are provided and supported by the partner, not by Dynatrace.

The trade-off to state plainly: you have added a dependency. The observability feed now inherits the pipeline's availability, its release cadence, and its owning team's change process. Where the pipeline is already business-critical this costs nothing; where it was a best-effort side system, direct ingestion may be the more honest choice.

> **Sources:** Cribl via HTTP (Dynatrace Hub) and Cribl via OpenTelemetry (Dynatrace Hub) — each reads *"Provided by Cribl"* and *"Support By the partner"*, Syslog via OpenTelemetry Collector (Dynatrace Hub).

3.2 Check the Hub before you build anything

Before designing a custom route, check whether the vendor already has a Dynatrace Hub listing. The answer materially changes the project:

- **A listing exists** — you inherit a documented path. Check the listing's *Provided by* and *Support by* lines to see whether Dynatrace or a partner stands behind it, then take it.
- **No listing exists** — you are building and owning the integration yourself. That is a legitimate and common position, but it should be a *decision*, not a discovery made in week three. It means you own the parsing, the field mapping, the failure modes, and the upgrade path when the vendor changes its schema.

In community practice, the second case is more common than teams expect for security and networking platforms specifically — verify for your own vendor rather than assuming either way.

3.3 Generic Collector receivers — and what to check in any of them

When no documented route fits a feed, a generic OpenTelemetry Collector receiver can sometimes fill the gap. Four are maintained in the `dynatrace-bindplane-otel-contrib` repository:

Receiver (v1.2.0)	Fills which row	Stability	Record time it sets
`restapi`	Pull only — polls a REST API with pagination, OAuth 2.0 client credentials and a checkpoint	beta	The first top-level `timestamp`, `time`, `created_at` … field, else the poll time

Receiver (v1.2.0)	Fills which row	Stability	Record time it sets
`httpreceiver`	HTTPS push — an HTTP endpoint the vendor posts to; it answers `200` only after the pipeline accepts the records	beta	Observed time only; the vendor's own time stays in the body
`azureblobpollingreceiver`	Files in Azure Blob Storage — polls a container for new blobs	beta	Observed time only, for NDJSON, text and `{"records": [...]}` blobs
`gcspubsubeventreceiver`	Files in Google Cloud Storage — reads each new object announced by a Pub/Sub notification	alpha	The time the object is read, for every line

All four ship only in the full Dynatrace Bindplane Distribution of the Collector (DBDOT). They are not in the slim DBDOT build or in the Dynatrace OTel Collector distribution, and Dynatrace's Collector support covers only the components that distribution includes. They configure in plain Collector YAML, so Bindplane is optional. Treat them as complements: where a documented Dynatrace route exists for a feed — direct ingest, ActiveGate syslog, the S3 log forwarder, a Hub extension — use it. FAQ-23 § 6.1 assesses `restapi` against Box, and FAQ-27 § 8.1 assesses the vendor-specific `m365` receiver against Microsoft 365.

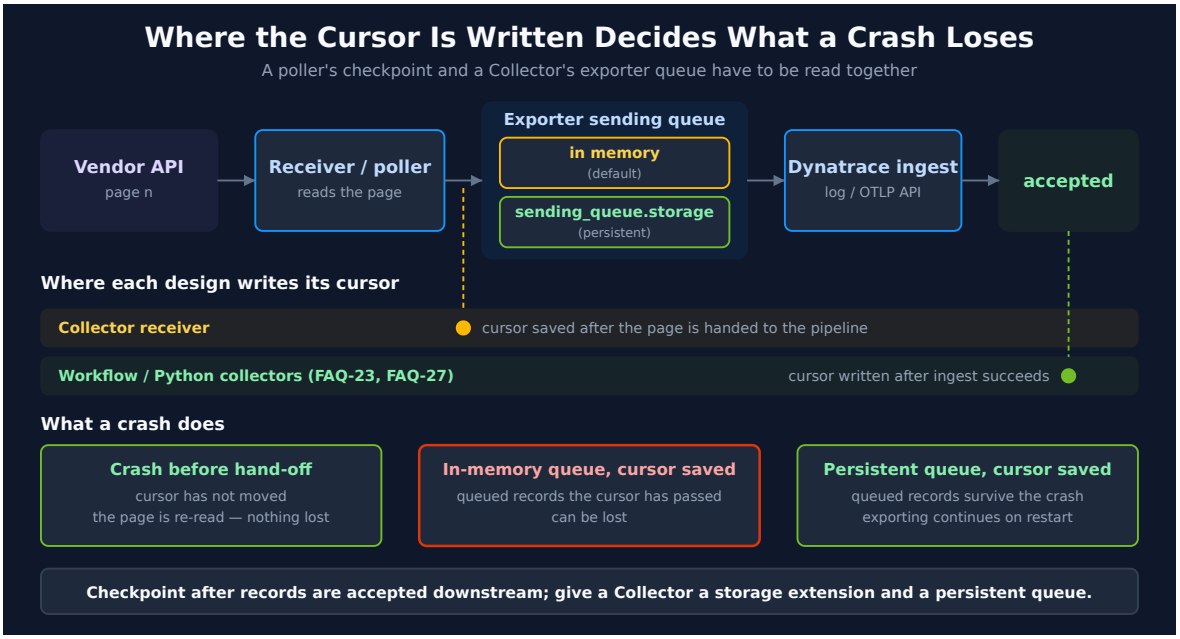
A receiver whose record time is not the event time needs a `transform` rule, or OpenPipeline parsing, before time-based queries mean anything.

What to check in any generic poller or receiver — a Collector component, a Workflow, or a script you wrote:

Check	Why it matters	Where it was found
Rate limits. Does it honour `429` and `Retry-After`, or treat them as errors?	A poller that ignores them keeps hitting the limit. One that treats them as errors must at least not move its cursor	`restapi` v1.2.0 treats every non-2xx as an error (FAQ-23 § 6.1)
Checkpoint after forwarding. Is the cursor written only after the records were accepted downstream, and is it durable — a storage extension or your own store?	A cursor written first loses a window on any failure. One held only in memory loses its place on every restart	`m365` v1.2.0 advances its checkpoint even when a listing fails (FAQ-27 § 8.1)
Persistent exporter queue. In a Collector, is `sending_queue.storage` set?	Without it the queue is in memory: records count as forwarded once queued, so a crash loses them after the checkpoint has moved	Any Collector-based collector
Last page and first run. What happens when the API returns no next cursor, and where does a first run start?	Dropping the cursor restarts the stream. An unseeded first run may replay everything the API still holds	`restapi` cannot seed an opaque cursor (FAQ-23 § 6.1); `m365` reads only the first page of a listing (FAQ-27 § 8.1)
Which timestamp. Does it stamp the vendor's event time, or the time it read the record?	Log ingest drops records older than 24 hours, so event time on a backlog loses it. Read time keeps it but places it at the wrong point on the timeline	The table above

Check	Why it matters	Where it was found
Duplicates. Does the API repeat records, and does anything dedupe them?	Repeats inflate counts and ingest cost	Box `admin_logs_streaming` (FAQ-23 § 4)

Where the cursor is written, and whether the exporter queue persists, decide what a crash costs:



Sources:

- OpenTelemetry Collector (DT docs) — *"This support covers only the components and their versions that are included in the Dynatrace OTel Collector."*
- restapireceiver README (Dynatrace GitHub), httpreceiver README (Dynatrace GitHub) — *"Anything that is able to send logs to an endpoint using HTTP will be able to utilize this receiver."*
- azureblobpollingreceiver README (Dynatrace GitHub), gcspubsubeventreceiver README (Dynatrace GitHub) — *"consumes GCS event notifications for object finalization events"*
- Log ingestion limits (DT docs) — *"The event is dropped if it contains a timestamp older than the configured log age limit." "For environments created after Dynatrace version 1.348 was released, the default log age limit is 72 hours." "For environments created before Dynatrace version 1.348 was released, the default log age limit is 24 hours. The limit can be increased to 72 hours on request."*
- Exporter helper (OpenTelemetry GitHub) — *"There is no in-memory queue when set."*; *"If the collector instance is killed while having some items in the persistent queue, on restart the items will be picked and the exporting is continued."*

> ˜ **Code read (10/29/2026):** at tag `v1.2.0`, each receiver's `metadata.yaml` gives the stability shown; `restapireceiver/logs.go` reads the record time from the first of `timestamp`, `time`, `created_at`, `createdAt`, `date`, `datetime`, `@timestamp`; `httpreceiver/logs.go` sets only the observed time and writes `200` after the pipeline accepts the records (`500` if it does not); `internal/blobconsume` sets only the observed time in its NDJSON, text, line and records-JSON consumers; `gcspubsubeventreceiver/internal/worker/worker.go` sets both times to the read time. Component lists: all four are in the full DBDOT manifest and none is in the slim DBDOT manifest or the Dynatrace OTel Collector manifest (0.59.0) (control: the full manifest lists `m365receiver`, which FAQ-27 reads the same way)
>
> ˜ **Derived:** the checklist generalizes the gaps found in FAQ-23 § 6.1 and FAQ-27 § 8.1; the queue row combines the exporter-helper sentence with a receiver forwarding records before it saves its cursor

---

4. Signal-First Ingestion — Extract, Then Discard

The default posture for a high-volume third-party feed should be: **process the record for its value, then throw the record away.** You keep the metric, the event, and the alert. You do not keep several hundred gigabytes of transaction records containing user identities and destinations that nobody will query and everybody will have to account for at audit time.

OpenPipeline supports this directly — but **the processor you choose, and the stage it sits in, decide whether it works or silently defeats itself.**

4.1 The stage order, and the trap inside it

Records traverse OpenPipeline stages in a fixed order — the one the docs list as *"ordered in the pipeline sequence of execution"*:

#	Stage	What you would use it for here
1	**Processing**	Parse, mask, hash, rename fields — and, if you choose, **Drop record**
2	**Smartscape node**	Calculate a Smartscape ID; optionally create/update the node (§ 5)

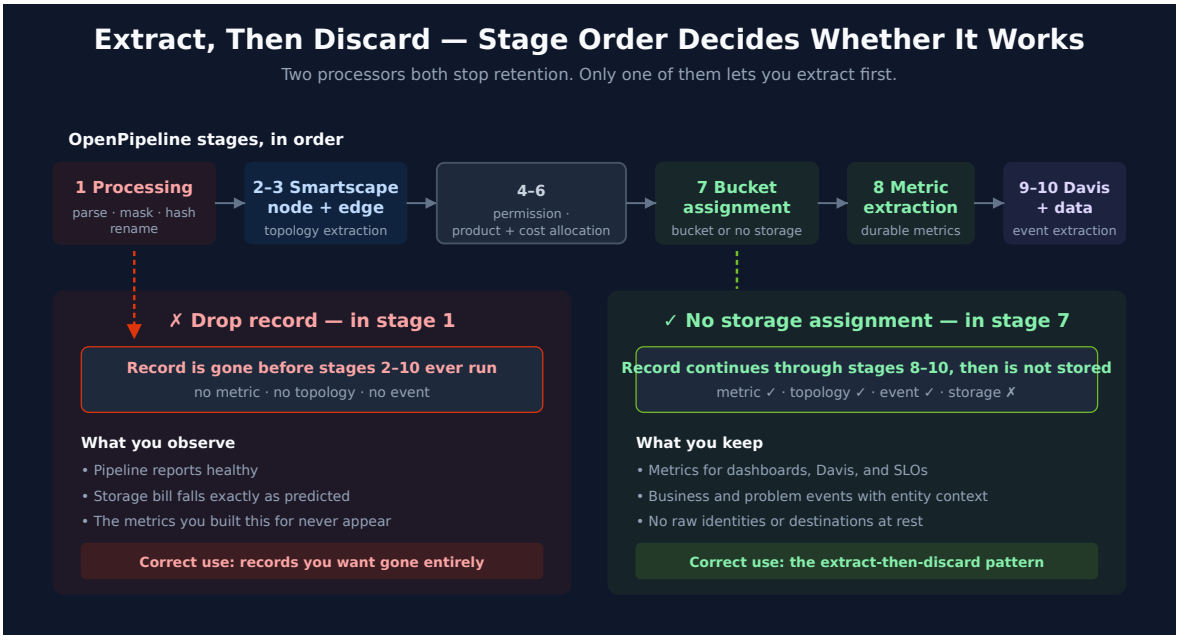
#	Stage	What you would use it for here
3	**Smartscape edge**	Record dynamic edges between nodes (§ 5)
4	**Permission**	Set record-level security context
5	**Product allocation**	Attribute consumption to a product
6	**Cost allocation**	Attribute consumption to a cost center
7	**Bucket assignment**	Assign a bucket — or **No storage assignment**
8	**Metric extraction**	Turn the record into a durable metric
9	**Davis**	Emit Davis events
10	**Data extraction**	Emit business events or SDLC events

Two processors both stop a record being retained, and they are **not** interchangeable:

Processor	Stage	Effect (docs, verbatim)
Drop record	Processing (1)	*"Drops a record. The record isn't processed further and isn't stored."*
No storage assignment	Bucket assignment (7)	*"Doesn't store the record after processing. The record continues through all configured pipeline stages and isn't stored only at the end of the pipeline."*

The trap: *Drop record* sits in stage 1 — **before** Smartscape extraction, metric extraction, Davis and data extraction. Use it for "extract then drop" and you drop the record before anything has been extracted from it. The pipeline reports healthy, the storage bill falls exactly as predicted, and the metrics you built the integration for never appear.

Note that metric extraction runs **after** bucket assignment. That ordering is safe only because *No storage assignment* does not end the record's journey — it marks the record as not-to-be-stored and lets it continue through every remaining stage.



Use `No storage assignment` in the Bucket assignment stage for the extract-then-discard pattern. The record runs the full gauntlet — parsed, metricized, topology-tagged, event-emitting — and is then simply not persisted. Reserve `Drop record` for records you want *gone*: health-check noise, a chatty debug category, a feed you enabled by mistake.

Sources:

- Processing in OpenPipeline (DT docs) — stage order and both processor definitions quoted above
- Configure data storage and retention for logs (DT docs) — *"the record continues through all configured pipeline stages and is not stored only at the end of the pipeline. This means you can extract metrics and generate alerts from records that you won't store."*
- Extraction stages in OpenPipeline (DT docs)
- Parse log lines and extract a metric (DT docs) — worked parse-and-extract-a-metric example; it has no discard step, so combine it with the *No storage assignment* step from the bucket-assignment page
- **Derived:** that `Drop record` in stage 1 pre-empts the extractors follows from its definition and the documented stage order; validate on a narrow route before applying it broadly

4.2 Decide what is worth keeping, by class

What you extract	Why it survives	Retention posture
Metric — a rate, a latency percentile, a saturation gauge, a count by dimension	It is what dashboards, anomaly detection, and SLOs actually read. Cheap to store, cheap to query	**Keep.** Metric reads are not billed as a query capability the way log scans are — see FAQ-09
Business event — a meaningful state change worth analyzing individually	Carries a handful of fields, not a full payload. Queryable, workflow-triggerable	**Keep**, with minimal fields — resist copying the whole record onto the event
Davis / problem event — a condition someone must act on	It is the thing that pages a human and reaches your incident tooling	**Keep** with entity context attached (§ 5)
Raw record	Forensics, incident reconstruction, compliance obligation	**Discard by default.** Retain only where security or compliance explicitly asks — and then in an isolated bucket (§ 6)

The economic argument is in **FAQ-09**: a metric you query on a dashboard every five minutes, forever, costs a fraction of the equivalent recurring log scan. A third-party feed that drives recurring dashboards and alerts is precisely the recurring-query case that rule exists for.

One caveat to state honestly. Extraction is **forward-only**. A metric starts existing the moment you configure it — it does not backfill from records already ingested, and it certainly does not backfill from records you dropped. If there is any chance you will want a dimension later, extract it from the start; adding it in month four gives you a metric with no history at the exact moment you need the comparison.

---

5. Modeling the Vendor's Objects as Topology

This is the section that separates an integration that ages well from one that becomes a pile of dashboards nobody trusts.

The vendor's world contains operationally meaningful objects: applications, connectors, gateways, regions, tenants, collectors. If those exist in Dynatrace only as *string fields on log records*, then every dashboard filter is a string match, every alert payload is a bag of text, and nothing you build participates in Smartscape. If they exist as **entities**, they get identity, lifetime, tags, relationships, and — crucially — they show up in alert context automatically.

The good news, and it is more recent than most teams realize: you no longer need the legacy custom-device API for this. OpenPipeline can extract Smartscape nodes and edges directly from ingested signals, including entirely custom entity types.

5.1 How it works

A **Smartscape ID** is "a unique identifier calculated from a node type and one or more ordered ID components" — you nominate the node type and which record fields form the identity, and the platform computes a stable ID (for example `CUSTOM_TRUCK-0123456789ABCDEF`, landing in a field like `dt.smartscape.custom_truck`).

Two behaviors sit behind one processor, and the difference matters:

- **ID enrichment only** — the processor calculates the ID and stamps it on the record. The record is now *associated with that entity in queries*, but no node is created. Use this on high-volume signals that reference an entity defined elsewhere.
- **Extract node enabled** — the processor additionally "creates or updates the node in Smartscape storage," including its configured fields and static edges. Use this on the signal that legitimately defines the object.

Naming is constrained: a custom node type "must begin with `CUSTOM_` or `EXT_`" and "must be unique in your environment." Pick the prefix scheme once, up front — renaming a node type later orphans every node already extracted under the old name.

Edges come in two kinds:

Edge kind	Behavior	Use for
Static	Stored on the node, inherits its lifetime, and overrides others of the same type when re-extracted	Structural facts: a connector *belongs to* a connector group; an application *is served by* a gateway
Dynamic	Recorded at a point in time; requires **both** source and target Smartscape IDs already present on the record	Observed, time-varying relationships: this gateway *served* this application during this window

The dynamic-edge prerequisite is the one that bites. Both IDs must already be on the record when the edge processor runs — which means you calculate them earlier, with node processors in the node stage, even where you do not want those processors to create nodes.

> **Sources:** Smartscape node and edge extraction in OpenPipeline (DT docs) — all quoted definitions, the `CUSTOM_` / `EXT_` naming constraint, and the static-vs-dynamic edge distinction, Define custom topology via OpenPipeline (DT docs) — the worked node/edge processor configuration this section generalizes, Smartscape on Grail (DT docs).

5.2 A modeling checklist

Before configuring processors, answer these five questions for each candidate object:

1. **Is it operationally meaningful?** Would somebody be paged about it, own it, or filter a dashboard by it? If not, it is a dimension, not an entity. Over-modeling is a real cost — every node type is a thing to maintain.
2. **What is its stable identity?** The ID components must be present on every record that references the object, and must not change when the object is renamed or moved. A hostname is often a poor choice; a vendor-assigned ID is usually a good one.
3. **Which feed defines it, and which merely mentions it?** Exactly one feed should extract the node. Everything else enriches with the ID only.
4. **What does it connect to?** Structural relationships are static edges; observed ones are dynamic.
5. **What is its lifetime?** Set *Node validity duration* (60–1440 minutes) to match how often the defining feed reports. A node that stops being extracted keeps its last `lifetime[end]` and drops out of any query whose timeframe starts after that — so absence checks need an explicit `from:` (§ 5.3).

> **Sources:** Smartscape node and edge extraction in OpenPipeline (DT docs) — *"Define how long the node should stay valid after the last update, from 60 to 1440 minutes."*, Smartscape on Grail (DT docs) — *"Based on the query timeframe, you will see only those nodes that have a lifetime overlapping with the query timeframe."*

5.3 Verifying the model

Once extraction is running, confirm the topology exists before building anything on it. Note that `smartscapeNodes` returns **zero rows rather than an error** for a node type that does not exist — a clean empty result is not proof your configuration is right, only that it is syntactically valid.

Discover which custom types actually landed:

```
// Which CUSTOM_* node types exist in this environment?
// The from: matters: without it only nodes seen in the last 2 hours are
returned,
// so a feed polled hourly or daily can look absent between runs. Set it to at
// least twice your poll interval.
// Zero rows is expected before the pipeline first runs, not an error. Confirm
it
// is really that, though: a zero here is also what a wrong node type looks
like.
smartscapeNodes "CUSTOM_*", from:-7d
| dedup type
| fields type
```

> Executed 10/02/2026: valid, zero rows on a tenant with no custom pipeline; the control `smartscapeNodes "AWS_*", from:-7d | dedup type` returned types, so the wildcard form works. Without `from:` the query's analysis timeframe was 2 hours.

Then inspect one type, and check for nodes that have stopped being refreshed:

```
// Inspect extracted nodes of one custom type, and keep only the live ones.
//
// lifetime[end] is a rolling "observed until" timestamp — it is ALWAYS
populated,
// even on a stale node, so it is not a tombstone. Verified 08/27/2026: of 12
HOST
// nodes, isNull(lifetime[end]) matched 0 and isNotNull matched all 12. Null-
testing
// it selects nothing, always; the inverse selects everything. Compare against
now().
smartscapeNodes CUSTOM_APP_CONNECTOR
| fieldsAdd last_seen = lifetime[end]
| filter last_seen &gt; now() - 10m
| fields id, name, last_seen
```

Finally, confirm the relationships resolve. Prefer the full `traverse` syntax with named parameters — the short positional form is accepted but far less readable, and a wrong edge type returns empty rather than erroring:

```
// Walk a custom relationship. Edge types are lowercase; node types are
uppercase.
//
// traverse REPLACES the record with the target node's own fields — id, name,
type.
// There is no sourceName or targetName: DQL resolves unknown identifiers to
null
// WITHOUT raising FIELD_DOES_NOT_EXIST, so selecting them returns the right
number
// of rows with every value empty and no error to tell you (verified
08/27/2026).
// A wrong edge type likewise returns zero rows with no error — check
smartscapeEdges
// if this comes back empty.
smartscapeNodes CUSTOM_SECURE_ACCESS_APP
| traverse edgeTypes: {served_by}, targetTypes: {CUSTOM_APP_CONNECTOR},
direction: forward
| fields id, name, type
```

---

6. Isolating What You Do Retain

Where § 4 concluded "retain this raw," the record needs somewhere to go that is not the default bucket.

Create a dedicated bucket for the vendor's retained records and assign permissions to it explicitly. Two independent reasons, and either alone justifies it:

- **Exposure.** These feeds carry user identities, destinations, device context, and policy outcomes. In the default bucket they are readable by everyone with general log-read access — which is usually a far wider group than the one entitled to see who visited what.
- **Retention economics.** A bucket carries its own retention period. Third-party transaction records typically want a *shorter* retention than application logs; without a dedicated bucket they inherit whatever the default is, indefinitely, at volume.

Bucket permissions are assigned per bucket through Account Management rather than through general environment roles — which is precisely the property that makes the

isolation meaningful.

Once the bucket exists, confirm the vendor's records are landing where you intended — group the vendor's feed by bucket, rather than counting a bucket's contents:

> **Sources:** Configure data storage and retention for logs (DT docs) — per-bucket retention and route-based storage assignment, Assign permissions in Grail (DT docs) — *"Permissions can be assigned at the bucket, table, record, and field level"*, which is what makes a dedicated bucket a real isolation boundary. **Derived:** the exposure argument for a dedicated bucket combines per-bucket permissioning with the identity content of these feeds; no page frames it as a vendor-feed rule.

```
// Where is the vendor's feed landing? Substitute the vendor's log.source
// (or whichever field your route matches on).
// Expect one row: your dedicated bucket. A default_logs row means the route
// or the bucket assignment is wrong.
// Every record in the window is evaluated, but the bytes scanned follow what
the
// filter matches: one hour read 2.1 GB unfiltered and 0.11 GB with a
selective
// log.source filter on the validation tenant (10/15/2026). A high-volume
source
// still reads its full volume, so keep the window short (see FAQ-09).
fetch logs, from:-1h
| filter log.source == "&lt;vendor-source&gt;"
| summarize records = count(), by:{dt.system.bucket}
```

> Executed 10/02/2026 with a real `log.source` from the validation tenant: one row per bucket, including `default_logs` for a source split across four buckets — the shape this check is meant to surface. One hour scanned 2.36 GB, no notifications. Re-run 10/15/2026: with the placeholder source the same hour scanned 0.11 GB against 2.1 GB unfiltered, so the bytes scanned follow the matched volume.

Complementary controls worth applying in the Processing stage, before storage:

Control	When to use it
Mask	The field's *shape* matters for debugging but its value does not

Control	When to use it
Hash	You need to correlate on a value across records without being able to read it — the usual answer for user identifiers
Field removal	The field has no analytical use at all. Cheapest control; use it first

Hashing deserves a caveat: it preserves correlation, which is the point, but a hash of a low-cardinality value is reversible by anyone who can enumerate the input space. Usernames drawn from a known directory are a good example. Hashing is a control against casual exposure, not against a determined analyst.

> **Sources:** Configure data storage and retention for logs (DT docs) — bucket assignment processor and per-bucket permission assignment via Account Management, Processing in OpenPipeline (DT docs). **Derived:** the hash-reversibility caveat is a general property of hashing low-cardinality inputs, not a Dynatrace-specific behavior.

---

7. Normalizing Dimensions Across Feeds

A vendor's feeds will disagree with each other about field names. The same application will be `appname` in one feed, `Application` in another, and `app_name` in the API response. If you carry those differences through to Grail, every cross-feed question becomes a join problem, and the correlation the integration exists to provide never materializes.

Normalize at ingest, in the Processing stage, once. Pick your target dimension names and rename into them for every feed. The set that matters is small:

Canonical dimension	Why it is load-bearing
`app`	The join key between every feed. Get this one right or nothing correlates
`user` (hashed, per § 6)	Ties experience to access to transaction
`location` / `region`	The most common triage axis, and the one executives ask about first

Canonical dimension	Why it is load-bearing
`department` / `business_unit`	Turns a technical dashboard into one a business owner will read
The vendor's own component identifiers	Feed the Smartscape ID components in § 5

Two rules make this durable:

- **Normalize to your names, not the vendor's.** Vendor field names change between product versions; your canonical names should not. The rename mapping is the thing that absorbs vendor churn.
- **Normalize before extraction, not after.** Metric dimensions and Smartscape ID components are computed from fields as they exist at that stage. Renaming afterwards leaves you with metrics dimensioned on the old names and no way to retrofit them.

For feeds that are not JSON, parsing happens here too — see **FAQ-15** for DPL, and note especially that DPL does not backtrack, which trips up patterns ported from regex.

> **Sources:** Processing in OpenPipeline (DT docs) — the Processing stage is where field renames happen, and it runs before the extraction stages, Extraction stages in OpenPipeline (DT docs), Smartscape node and edge extraction in OpenPipeline (DT docs) — ID components are read at extraction time, which is what makes normalize-before-extract load-bearing. **Derived:** the canonical-dimension list is this entry's recommendation, not a documented set.

8. Where This Plugs Into the Rest of the Corpus

Once the data is in Grail, normalized, and carrying topology, **nothing downstream is special.** That is the point of the pattern, and it is why this entry stops here rather than continuing into dashboards. Consume it the way you consume everything else:

What you want to build	Where it is documented	What the work in this entry buys you
Dashboards	`DASH` series	Entities from § 5 become filter dimensions and drill-down targets rather than string matches
Alerting and routing	`ALERT` series — ALERT-01 architecture, ALERT-02 choosing detection, ALERT-03 routing and cost, ALERT-04 ServiceNow	Alerts raised against entities carry topology into the payload automatically, which is what makes downstream correlation and routing work
SLOs and error budgets	`SLO` series — SLO-02 for SLI definition, SLO-04 for burn-rate alerting	Extracted metrics (§ 4) are exactly the shape an SLI wants
Anomaly detection	`AIOPS` series	Davis works on the extracted metrics; it cannot work on records you dropped, which is another argument for extracting generously in § 4
Synthetic validation	`SYNTH` series	Useful for testing a path the vendor's own telemetry cannot see — but check § 2 first; the vendor's experience feed often already answers the question
Bucket and retention design	`ORGNZ` series — ORGNZ-02 for buckets	§ 6 is the summary; ORGNZ is the depth
Consumption and cost	`FINOPS` series	Third-party feeds are a common source of unplanned ingest growth — FINOPS-02 for forecasting, FINOPS-03 for the Cut/Tune/Filter framework
Query economics	`FAQ-09`	The metric-versus-log-scan reasoning behind § 4

On external consumers. Where an external tool — an event-correlation platform, an ITSM system, an SLO product — is the destination for this data, the work in § 5 is what makes that integration good rather than noisy: a payload carrying an entity, its relationships, and its business context beats a payload carrying a log line. Dynatrace-native equivalents exist for each (Workflows and problem notifications for routing, the SLO

app for objectives), and it is worth knowing which you are choosing and why. ALERT-03 covers the routing-and-cost trade-off directly.

---

9. Recommended Approach

A defensible sequence. The ordering is deliberate — each step's output is the next step's input, and steps 1 and 2 are the ones teams skip and later regret.

#	Step	Output
1	**Write down the questions.** Five operational questions the integration must answer, each mapped to a signal class (§ 2)	A scope you can defend when someone asks for "all the logs"
2	**Inventory the feeds and their transports.** For each: format, transport, volume estimate, sensitivity, and whether it already flows somewhere (§ 3)	A route decision per feed, not one blanket route
3	**Check the Hub.** Determine whether you are adopting a supported path or owning a custom one (§ 3.2)	An explicit, early decision about who owns the integration
4	**Get the retention decision signed off.** Which feeds may be retained raw, for how long, with which fields masked or hashed (§ 4, § 6)	Security approval *before* the pipeline exists
5	**Design the topology model.** Objects, ID components, edges — on paper, using the § 5.2 checklist	A node/edge design you can review before it is expensive to change
6	**Build one feed end to end.** Route → normalize → extract → topology → discard. One feed, fully finished	A working reference implementation, and a realistic estimate for the rest
7	**Verify.** Confirm nodes and edges exist (§ 5.3), metrics are populating, and records are landing in the intended bucket (§ 6)	Evidence, not assumption
8	**Then scale out**, and only then build dashboards, alerts, and SLOs on top (§ 8)	Downstream work that inherits topology from day one

The single highest-value decision is step 5. Extraction and routing can be reworked cheaply. A topology model that nobody designed — where the vendor's objects live on as string fields — is what leaves you, two quarters later, with dashboards that filter on text and alerts that carry no context.

---

10. Common Gotchas

#	Gotcha	What to do
1	`Drop record` **used for extract-then-discard.** It sits in the Processing stage, ahead of every extractor — the record is gone before a metric is made from it	Use **No storage assignment** in the Bucket assignment stage. `Drop record` is for records you want gone entirely (§ 4.1)
2	**Assuming every feed can go direct.** A vendor commonly uses HTTPS for one product line and raw TCP for another; only the first can reach the ingest API unaided	Route per feed, from the transport (§ 3)
3	**Extraction is forward-only.** A metric added in month four has no history in months one through three	Extract the dimensions you might want from the start (§ 4.2)
4	**Dynamic edges silently produce nothing** when both Smartscape IDs are not already on the record	Calculate both IDs with node processors in the node stage first, even where you do not want them to create nodes (§ 5.1)
5	**A wrong node or edge type returns zero rows, not an error.** Empty results look identical to "not configured yet"	Verify with `smartscapeNodes "CUSTOM_*", from:-7d` (a `from:` wider than your poll interval) and `smartscapeEdges` before concluding the pipeline is broken (§ 5.3)
6	**Renaming a custom node type orphans existing nodes.** The type is part of the ID	Settle the `CUSTOM_` / `EXT_` naming scheme before the first extraction (§ 5.1)
7	**Normalizing after extraction.** Metric dimensions and ID components are computed from fields as they stand at that stage	Rename in the Processing stage, ahead of everything (§ 7)

#	Gotcha	What to do
8	**Retaining raw into the default bucket** because retention was never explicitly decided	Dedicated bucket with its own permissions and retention, or discard (§ 6)
9	**Hashing a low-cardinality identifier and calling it anonymized.** A hashed username from a known directory is enumerable	Hash for correlation, not as a privacy guarantee; remove the field where you do not need to correlate on it (§ 6)
10	**Shipping only the access logs** because they were the easy feed, then discovering they cannot answer experience questions	Map questions to signal classes first (§ 2)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.